in the following listing), the first one will be resolved, unless you inject an `IEnumerable`.

Listing 7.15 `TryAdd<LIFETIME>` resulting in the first implementation being resolved

```
builder.Services.TryAddScoped<IPriceService, FrPriceService>();     ❶
builder.Services.TryAddScoped<IPriceService, GbPriceService>();
builder.Services.TryAddScoped<IPriceService, UsPriceService>();
builder.Services.TryAddScoped<IPriceService, DefaultPriceService>();
```

❶ The implementation that was registered first is resolved.

So when would you use the `TryAdd` method for registering services? Typically, you would use this approach if you wanted to ensure additional registrations made accidentally are not used by default. This could happen if it is not clear what registrations are being made because they are hidden within an extension method—for example, the `AddRazorPages` method. The library authors might want to ensure their registration is used regardless of what consumers of the framework subsequently try to do.

7.3 Other ways to access registered services

Constructor injection is likely to be the most common way you work with registered services. However, there are other ways to access services you should be aware of. You will likely use some of these options at some stage, but they have their caveats. The options include injecting directly into Razor files, method injection, and retrieving services directly from the service container.

7.3.1 View injection